

THE ML LIFECYCLE

1. **Frame** the problem: decision, metric, cost of errors, constraints (latency, explainability).
 2. **Data**: collect, label, validate, version.
 3. **Features** and a **baseline**; train and evaluate with the right split.
 4. **Package and deploy**: batch job or online service.
 5. **Monitor**: system health, data quality, drift, model performance, business impact.
 6. **Retrain / improve** and release safely; **retire** old models.
- It is a loop: monitoring sends you back to data and features. Most effort is data and operations, not model code. A model in a notebook has no value until it is deployed and monitored.
 - **Version everything**: code (Git), data (DVC, lakeFS, snapshots), features, model artefacts and config, environment (Docker, pinned deps) so any prediction can be reproduced.

SERVING PATTERNS

Pattern	How	Use when
Batch	score on a schedule, store results	recommendations refreshed daily, risk lists; simplest
Online (real-time)	API returns a prediction in ms	fraud at checkout, search ranking
Streaming	score events as they arrive	monitoring, alerts
Edge / on-device	model runs on phone or device	offline, privacy, latency

- Online serving: wrap the model in an API (FastAPI/Flask/BentoML/TorchServe), containerise with Docker, run behind a load balancer on Kubernetes or a managed endpoint (SageMaker, Vertex AI). Concerns: **latency** (p50/p95/p99), throughput, autoscaling, cold starts, timeouts, fallbacks, caching, batching requests, cost.
- Reduce latency: smaller model, distillation, quantisation, ONNX, precomputed features, GPU only if needed.

```
from fastapi import FastAPI
import joblib
app = FastAPI(); model =
joblib.load("m.pkl")
@app.post("/predict")
def predict(x: dict):
    p = model.predict_proba([[x["a"],
x["b"]]])
    return {"score": float(p[0][1])}
```

TRAINING-SERVING SKEW AND FEATURE STORES

- **Skew**: features computed differently in training (SQL on history) and serving (service code), or with data not available at serving time. Causes silent accuracy loss.
- Prevent: **one shared feature definition (feature store)**: Feast, Tecton, Databricks) with an offline store for training and an online store for low-latency serving; point-in-time-correct joins; log served features and compare with training; same preprocessing code in a versioned pipeline artefact.
- Training data must reflect what was known **at prediction time** (no future data), including late-arriving labels.

DRIFT AND DECAY

Type	Meaning	Detect
Data (covariate) drift	input distribution P(X) changes	PSI, KS test, chi-square, summary stats
Label (prior) drift	outcome rate changes	label rate over time
Concept drift	relationship P(y given X) changes	performance drop with labels; error by segment
Upstream change	schema, units, missing rates, new categories	data-quality checks

- **PSI** = $\sum (\text{actual}\% - \text{expected}\%) \times \ln(\text{actual}\% / \text{expected}\%)$ per bucket. Rule of thumb: < 0.1 stable; 0.1-0.25 moderate shift; > 0.25 major shift. Example: shares 50/30/20 → 40/30/30 gives PSI = 0.063.
- Causes: seasonality, product changes, new users, fraud adapting, pipeline bugs. Models decay: plan for it.

MONITORING LAYERS

Layer	What to track
System	latency, errors, throughput, CPU/GPU, memory, cost
Data	schema, missing/null rate, ranges, new categories, freshness, volume
Model	prediction distribution, drift (PSI/KS), confidence, performance on labels
Business	conversion, revenue, complaints, fairness by group

- **Delayed labels** (fraud chargebacks, churn): monitor input and prediction drift and proxy metrics now; compute true performance when labels arrive (evaluate on matured cohorts).
- Alerts with owners and runbooks; dashboards; log inputs, outputs, model version and request id (with privacy controls); sample for human review.

SAFE RELEASE STRATEGIES

- **Shadow mode:** new model scores live traffic silently; compare with current.
- **Canary:** route a small share (1-5%) to the new model; watch metrics; ramp or **roll back**.
- **A/B test:** randomise users, measure the business metric; **interleaving** for ranking. Bandits for fast learning.
- **Blue/green:** two environments, switch traffic, instant rollback. Feature flags for kill switches.
- Always keep the previous model deployable. Validate a new model against the champion on a holdout **before** release (champion/challenger).

RETRAINING

- Triggers: schedule (weekly), drift alert, performance drop, new data volume, business change.
- Retrain pipeline: fetch data → validate - > features → train → evaluate vs champion → register → deploy via canary. Automated but gated.
- **Continuous training (CT)** with CI/CD: tests on code and data, build image, deploy staging, promote.
- Online learning is rarer; beware feedback loops (the model changes the data it later trains on) and label bias.

CI/CD FOR ML AND TESTING

- **CI:** unit tests (feature code), data-validation tests (schema, ranges, no leakage), model tests (beats baseline, no regression on slices, invariance/directional tests), integration test of the API.
- **CD:** build the image, run smoke tests, deploy to staging, promote to production with approval; infrastructure as code.
- Registry stages: **None → Staging → Production → Archived**; each model links to code commit, data version, metrics and owner (**MLflow Model Registry**, SageMaker, Vertex).

EXPERIMENT TRACKING AND REPRODUCIBILITY

```
import mlflow
with mlflow.start_run():
    mlflow.log_params({"lr": 0.1,
"depth": 6})
    mlflow.log_metric("auc", 0.87)
    mlflow.sklearn.log_model(model,
"model")
```

- Track parameters, metrics, artefacts, code version, data version, environment. Fix random seeds; pin dependencies; containerise.
- **Model card:** purpose, data, metrics by segment, limits, risks, owner. Data card for datasets.

KUBERNETES AND SCALING BASICS

- **Pod** = one or more containers; **Deployment** keeps N replicas and does rolling updates; **Service** gives a stable address; **Ingress** routes external traffic; **HPA** autoscales on CPU/QPS; **readiness/liveness probes** gate traffic and restart failures.
- Resource requests/limits (CPU, memory, GPU); node pools for GPU; model files from object storage or baked into the image.
- SLOs: availability (99.9%), latency (p95 < 200 ms), error budget. Load-test before launch; add timeouts, retries with backoff, circuit breakers, and a rule-based fallback.

LLMOPS (GENAI IN PRODUCTION)

- Version **prompts**, retrieval settings and model names like code; run an **evaluation set** on every change (faithfulness, correctness, format validity, safety).
- Monitor token usage, cost per request, latency, refusal and error rates, user feedback (thumbs), hallucination reports; log prompts and outputs with PII controls.

- **Guardrails:** input and output filters, schema validation, tool allow-lists, rate limits, prompt-injection defences; cache frequent answers; route easy queries to a smaller model.
- Vector index freshness and re-embedding when the embedding model changes.

RESPONSIBLE ML AND SECURITY

- **Fairness:** compare error rates and outcomes across groups; remove proxies carefully; document trade-offs. **Explainability:** SHAP, reason codes (regulated lending). **Privacy:** minimise data, anonymise, access control, consent; **adversarial** inputs and model extraction; prompt injection for LLMs.
- Human in the loop for high-stakes decisions; fallback rules when the model is down or uncertain.

INCIDENT RUNBOOK: ACCURACY DROPPED

1. Confirm it is real (metric definition, delayed labels, volume).
2. Check data quality and upstream changes (schema, null rates, units).
3. Compare feature and prediction distributions with training; find the shifted features.
4. Check recent releases (code, model, config) and segments.
5. Mitigate: roll back, fall back to rules, retrain; then fix the root cause and add the missing alert.

SAY IT IN 20 SECONDS

- "Batch is simple; online is for real-time needs."
- "Skew means training and serving compute features differently; a feature store fixes it."
- "Data drift is inputs changing; concept drift is the relationship changing."
- "Ship with shadow or canary and a rollback."
- "Labels arrive late: watch drift and proxies now, performance later."